

SOLID Principles

Java — Simplest Examples

Sub-Chapter 02 · 01

S	Single Responsibility	One class, one job
O	Open/Closed	Open to extend, closed to modify
L	Liskov Substitution	Subtypes must honour the contract
I	Interface Segregation	Small, focused interfaces
D	Dependency Inversion	Depend on abstractions, not concretions

S Single Responsibility Principle

One class should have only one reason to change

A class should do **one thing**. If a class handles two concerns, changing one concern risks breaking the other.

Simplest Example: Invoice

We have an **Invoice**. Two separate concerns: calculating the total, and printing/saving it. SRP says: split them.

■ WRONG — two jobs in one class

```
class Invoice {
    double total() { return qty * price
; }

    // second responsibility: printing
    void print() {
        System.out.println("Invoice: "
+ total());
    }

    // third responsibility: saving
    void saveToFile() {
        Files.write(Path.of("inv.txt"),
            total().toString().getBytes
());
    }
}
```

■ RIGHT — one job each

```
class Invoice {
    double total() { return qty * price
; }
}

class InvoicePrinter {
    void print(Invoice inv) {
        System.out.println(inv.total())
;
    }
}

class InvoiceSaver {
    void save(Invoice inv) {
        Files.write(Path.of("inv.txt"),
            String.valueOf(inv.total())
                .getBytes());
    }
}
```

Key Rules

- Ask: 'Who would ask me to change this class?' — there should be only ONE answer.
- If a class has 'and' in its description (calculates AND saves), split it.
- SRP does not mean one method — a class can have many methods for the same concern.

Q1. What is the Single Responsibility Principle?

A class should have only one reason to change — meaning it should own exactly one concern or responsibility.

Q2. How do you detect an SRP violation?

If you can describe the class with 'and' (it calculates AND saves AND logs), it has multiple responsibilities.

Q3. Does SRP mean every class has one method?

No. A class can have many methods as long as they all serve the same single concern.

Q4. What is a 'reason to change'?

A stakeholder or requirement axis. Business rules change independently of persistence, which changes independently of UI.

Q5. Give a real-world analogy.

A chef cooks; a cashier handles billing. They are separate roles — one person doing both is an SRP violation.



Open/Closed Principle

Open for extension, closed for modification

You should be able to add new behaviour **without editing existing, tested code**. Use abstraction (interface/abstract class) so new cases are added by creating new classes.

Simplest Example: Shape Area

We want to calculate the area of different shapes. Each time we add a shape we must NOT touch the calculator.

■ WRONG — must edit every time

```
class AreaCalc {
    double calc(Object shape) {
        if (shape instanceof Circle c)
            return Math.PI * c.r * c.r;
        if (shape instanceof Square s)
            return s.side * s.side;
        // add Triangle? edit this file again!
        throw new IllegalArgumentException(
        );
    }
}
```

■ RIGHT — add Triangle without edits

```
interface Shape {
    double area();
}

class Circle implements Shape {
    double r;
    public double area() {
        return Math.PI * r * r;
    }
}

class Square implements Shape {
    double side;
    public double area() {
        return side * side;
    }
}

// Adding Triangle = new class only
class Triangle implements Shape {
    double b, h;
    public double area() { return 0.5*b*h; }
}

class AreaCalc {
    double calc(Shape s) { return s.area(); }
}
```

Key Rules

- Introduce an abstraction (interface/abstract class) for the varying behaviour.
- New variants are new classes; existing code is never changed.
- The pattern most associated with OCP is the Strategy Pattern.

Q1. What does 'closed for modification' mean?

Once a class is written and tested, you should not need to change its source code to add new features.

Q2. What does 'open for extension' mean?

You can add new behaviour (new shapes, new strategies) by writing new classes that implement an existing interface.

Q3. How does OCP relate to the Strategy Pattern?

Strategy encapsulates varying algorithms behind a common interface — you add a new algorithm without touching the caller.

Q4. Is OCP always about interfaces?

No. Inheritance (abstract class + subclass) also works, but interfaces are more flexible.

Q5. Give a real-world analogy.

A power strip: you add new devices (extension) by plugging them in — you never rewire the strip (modification).



Liskov Substitution Principle

Subtypes must be substitutable for their base type

If **S** is a subtype of **T**, you must be able to replace **T** with **S** everywhere **without breaking the program**. A subclass must honour the *contract* of its parent.

Simplest Example: Bird / Penguin

A Bird can fly. Penguin IS-A Bird but cannot fly — it violates LSP.

■ WRONG — Penguin breaks the contract

```
class Bird {
    void fly() {
        System.out.println("I can fly!")
    };
}

class Penguin extends Bird {
    @Override
    void fly() {
        // Penguins can't fly!
        throw new UnsupportedOperationException();
    }
}

// Caller breaks at runtime:
Bird b = new Penguin();
b.fly(); // BOOM
```

■ RIGHT — separate flying from non-flying

```
interface Bird { void eat(); }

interface FlyingBird extends Bird {
    void fly();
}

class Sparrow implements FlyingBird {
    public void eat() { ... }
    public void fly() {
        System.out.println("Flying!");
    }
}

class Penguin implements Bird {
    public void eat() { ... }
    // no fly() – and that is fine
}

// Safe substitution:
Bird b = new Penguin();
b.eat(); // works perfectly
```

Key Rules

- A subclass must not throw exceptions the parent does not throw.
- A subclass must not weaken preconditions or strengthen postconditions.
- Prefer interface segregation over deep inheritance to avoid LSP traps.
- If a subclass overrides a method with 'throw new UnsupportedOperationException()', LSP is violated.

Q1. What does LSP say in one sentence?

Objects of a subclass must behave correctly wherever the base class is expected — no surprises.

Q2. What is the classic LSP violation example?

Square extends Rectangle: setWidth on a Square also changes height, breaking code that expects only width to change.

Q3. How does LSP relate to contracts?

A contract is the set of pre/post-conditions and invariants. The subclass must honour all of them.

Q4. How do you fix an LSP violation?

Either redesign the hierarchy (use interfaces) or use composition instead of inheritance.

Q5. Does LSP apply to interfaces?

Yes — any implementation of an interface must fulfil the interface's implied contract, not just its method signatures.

I Interface Segregation Principle

Clients should not be forced to depend on methods they don't use

A **fat interface** forces all implementors to provide methods they don't need. Split it into small, focused interfaces.

Simplest Example: Printer / Scanner

An old printer can only print. Forcing it to implement scan() and fax() is an ISP violation.

■ WRONG — fat interface

```
interface Machine {
    void print(Document d);
    void scan(Document d);
    void fax(Document d);
}

// Simple printer forced to stub:
class OldPrinter implements Machine {
    public void print(Document d) {
        // real work
    }
    public void scan(Document d) {
        throw new UnsupportedOperationException();
    }
    public void fax(Document d) {
        throw new UnsupportedOperationException();
    }
}
```

■ RIGHT — small interfaces

```
interface Printer {
    void print(Document d);
}

interface Scanner {
    void scan(Document d);
}

interface Fax {
    void fax(Document d);
}

// OldPrinter only implements what it needs:
class OldPrinter implements Printer {
    public void print(Document d) {
        // real work
    }
}

// Modern all-in-one implements all:
class AllInOne
    implements Printer, Scanner, Fax {
    public void print(Document d) {...}
    public void scan(Document d) {...}
    public void fax(Document d) {...}
}
```

Key Rules

- If an implementation throws `UnsupportedOperationException`, the interface is probably too fat.
- Split by role: who uses which methods? Each role gets its own interface.
- ISP improves testability: mock only the interface you care about.

Q1. What is a 'fat interface'?

An interface that groups unrelated methods, forcing all implementors to provide methods they don't actually need.

Q2. How is ISP different from SRP?

SRP applies to classes (one reason to change). ISP applies to interfaces (clients should not see methods irrelevant to them).

Q3. What is the symptom of an ISP violation?

Implementations that stub or throw for several interface methods — they were forced to implement things they don't need.

Q4. Does ISP mean every interface has one method?

No. Methods that always go together belong in the same interface. Split only when clients need different subsets.

Q5. Give a real-world analogy.

A TV remote with 50 buttons: most users only need power, volume, channel. A simpler remote (ISP) removes unused buttons.

D Dependency Inversion Principle

Depend on abstractions, not on concretions

High-level modules should not import low-level modules directly. Both should depend on an **interface**. This makes it easy to swap implementations (e.g. switch databases, notification channels).

Simplest Example: Light Switch / Bulb

A switch should not be hard-wired to a specific bulb brand. It should depend on a Switchable interface.

■ WRONG — switch knows the bulb

```
class PhilipsBulb {
    void turnOn() { System.out.println
("Philips ON"); }
    void turnOff() { System.out.println
("Philips OFF"); }
}

class Switch {
    // HIGH-level depends on LOW-level
    directly
    private PhilipsBulb bulb = new PhilipsBulb();

    void operate(boolean on) {
        if (on) bulb.turnOn();
        else    bulb.turnOff();
    }
}
// Changing to OsramBulb means editing
Switch!
```

■ RIGHT — switch depends on abstraction

```
interface Switchable {
    void turnOn();
    void turnOff();
}

class PhilipsBulb implements Switchable {
    {
        public void turnOn() { ... }
        public void turnOff() { ... }
    }
}

class OsramBulb implements Switchable {
    public void turnOn() { ... }
    public void turnOff() { ... }
}

class Switch {
    // depends on interface, not concrete class
    private Switchable device;
    Switch(Switchable device) {
        this.device = device;
    }
    void operate(boolean on) {
        if (on) device.turnOn();
        else    device.turnOff();
    }
}

// Swap bulb without touching Switch:
Switch s = new Switch(new OsramBulb());
```

Key Rules

- Inject dependencies through constructor (constructor injection) — most visible and testable.
- The interface belongs to the high-level module, not the low-level one.
- DIP is the foundation for Dependency Injection frameworks (Spring, Guice).
- In tests you inject a mock/stub instead of a real implementation.

Q1. What are 'high-level' and 'low-level' modules?

High-level = business logic (Switch). Low-level = detail/implementation (PhilipsBulb). DIP says HI depends on an abstraction, not the detail.

Q2. What is Dependency Injection (DI)?

A technique where dependencies are passed in (injected) rather than created inside the class. Constructor injection is the most common form.

Q3. How is DIP different from DI?

DIP is the principle (depend on abstractions). DI is a pattern/technique used to achieve it.

Q4. How does DIP improve testing?

You can inject a fake/mock Switchable in tests without instantiating a real bulb or database.

Q5. Give a real-world analogy.

An electrical socket is an abstraction. Any compliant plug (phone charger, lamp) works. The wall does not know which device is plugged in.

SOLID Quick-Reference Cheat Sheet

Principle	One-liner	Violation symptom	Fix
SRP	One class, one job	Class has 'and' in its description	Extract into separate classes
OCF	Add, don't edit	Every new variant requires if/else edits	Introduce interface + new subclass
LSP	Subtypes keep the contract	Subclass throws <code>UnsupportedOperationException</code>	Redesign hierarchy / use composition
ISP	Small, focused interfaces	Implementations stub unused methods	Split fat interface by client role
DIP	Depend on abstractions	High-level class directly instantiates low-level	Inject via interface / constructor

Memory Aid: **S**orting **O**ld **L**egacy **I**s **D**ifficult

Key Takeaway

SOLID is not about memorising rules — it is about **managing change**. Every principle reduces the ripple effect of a single change so your codebase stays maintainable as it grows.